

Video Script

VIDEO RECORDING SCRIPT

Renewal Risk Intelligence Engine — Demo Walkthrough

SETUP BEFORE RECORDING

Open these windows BEFORE you hit record: 1. VS Code with the project open 2. Terminal (PowerShell) ready at the project folder 3. Browser tab ready for Streamlit (will open automatically) 4. GitHub repo page: <https://github.com/sohammmmm10/renewal-intelligence>

PART 1: INTRODUCTION (0:00 - 1:00)

| [SHOW: GitHub repo page or README in VS Code]

SAY:

“Hi, I’m Soham. I’ll walk you through my solution for the Renewal Risk Intelligence Engine assignment.

So what is this project? In one line — it’s a tool that predicts which SaaS customers are likely to cancel their subscription, and more importantly, it explains WHY and WHAT to do about it.

Let me give you a quick business context. Imagine you’re the BizOps team at Contentstack. You have 120 customer accounts, each paying anywhere from 17 thousand to 1.9 million dollars per year. Their contracts are coming up for renewal in the next 90 days. The question is — which of these customers are at risk of leaving? And where should your team spend their limited time?

Currently, most teams rely on gut feeling, scattered Slack messages, and last-minute panic. My solution replaces that with a data-driven, AI-powered system that reads ALL the signals, scores every account, and writes plain-English explanations that any salesperson can understand.”

PART 2: THE DATA — WHAT WE'RE WORKING WITH (1:00 - 2:30)

| [SHOW: The data/ folder in VS Code, click through each file briefly]

SAY:

“Let me show you the data we’re working with. We have 5 data sources plus a product changelog — and each one tells a different part of the story.

First — accounts.csv. This is our master list — 120 customers with their name, how much they pay, when their contract ends, and what plan they’re on.

Second — usage_metrics.csv. This has 6 months of product usage for each account — how many API calls they made, how many users are active, how much content they created, and importantly — what SDK version they’re running. This gives us 720 rows of data.

Third — support_tickets.csv. About 270 support tickets — we can see which are critical P1 issues, which are still open, and how long they took to resolve.

Fourth — nps_responses.csv. These are customer satisfaction surveys. Score of 0 to 10 — where 0 to 6 means unhappy, 7-8 means neutral, and 9-10 means happy. But here’s the interesting part — some comments are in Chinese, French, and Spanish. And some scores CONTRADICT their comments.

Fifth — csm_notes.txt. This is the goldmine. These are messy, handwritten notes from Customer Success Managers — the people who actually talk to customers. These notes mention competitors by name, internal politics, champion losses, billing disputes. But the formatting is all over the place — different date formats, misspelled names, inconsistent separators.

And finally — changelog.md. This is the product release log. Hidden inside it is a critical fact — SDK version 3 is being deprecated and loses security patches on April 30th, 2026. Any customer still on v3 is being forced to either migrate or leave.”

PART 3: THE ARCHITECTURE — HOW IT WORKS (2:30 - 4:30)

| [SHOW: README.md — scroll to the Architecture diagram]

SAY:

“Now let me walk you through how my pipeline works. I designed it as a 6-step process.

Step 1 is Data Ingestion. I load all 5 data sources. The CSVs are straightforward, but the CSM notes required custom regex parsing because every note has a different format — some start with ‘Mar 12 dash Acme Corp’, others say ‘2026-03-20 pipe NovaTech pipe James’. I built multiple regex patterns to handle all these variations.

Step 2 is Name Reconciliation. This is where it gets interesting. The CSM notes have typos — ‘BritePath Solutions’ when the real name is ‘BrightPath Solutions’. ‘Pinnacle Media’ instead of ‘Pinnacle Media Group’. ‘Thunderbolt Moters’ with a missing letter. I use fuzzy matching with a library called RapidFuzz to match these misspelled names to the correct accounts. I successfully matched 26 out of 27 notes.

Step 3 is Signal Extraction — the core of the system. I built 6 independent detectors, each producing a risk score from 0 to 1:

- Usage Decline — are they using the product less? I compare the first 2 months average to the last 2 months average across API calls, active users, content creation, and workflows.
- Support Tickets — do they have a lot of unresolved P1 tickets? I look at the high-priority ratio, open ticket ratio, and resolution time.
- NPS Score — what did they say in the satisfaction survey? I also translate non-English comments using GPT and detect score-sentiment mismatches.
- CSM Sentiment — this is LLM-powered. I send each CSM note to GPT-4o-mini and ask it to extract sentiment, competitor mentions, champion status, and key concerns. No regex can understand phrases like ‘never a good sign’ or ‘classic silent churn pattern’ — you need an LLM for this.
- SDK Deprecation Risk — I cross-reference the changelog with usage data. Any account on SDK v3 gets a high risk score because they’re facing a forced migration deadline.
- Engagement — are they actually using what they’re paying for? If an Enterprise account has only 32 active users when we’d expect 80, that’s shelfware — and a downgrade risk.

Step 4 is Risk Scoring. I combine all 6 signals using a weighted formula. Usage and CSM sentiment each get 25% weight because they’re the strongest predictors. SDK risk and tickets get 15% each. NPS and engagement get 10% each. The final score maps to three tiers — High risk is 65% and above, Medium is 40-65%, and Low is below 40%.

Step 5 is LLM Explanations. For every High and Medium risk account, I send all the computed signals to GPT and ask it to write a 3-to-5 sentence briefing explaining what’s wrong and what to do about it. This turns raw numbers into actionable intelligence.

Step 6 is Non-Obvious Insights. This is what the assignment specifically asked for — patterns that a simple rule-based system would miss. I’ll show you the actual results in a minute.”

PART 4: LIVE DEMO — RUNNING THE PIPELINE (4:30 - 7:00)

```
[SHOW: Terminal — run the pipeline] TYPE: .\env\Scripts\activate
TYPE: python run_pipeline.py
```

SAY:

“Let me run the pipeline live. I’ll activate the virtual environment and run the main script.

(wait for Step 1) You can see Step 1 loaded 120 accounts, 720 usage rows, 271 tickets, 98 NPS responses, and 27 CSM notes.

(wait for Step 2) Step 2 — fuzzy matching reconciled 26 out of 27 CSM notes. That means messy names like ‘BritePath’ got correctly matched to ‘BrightPath Solutions’.

(wait for Step 3) Now it’s extracting signals. Usage, tickets, and SDK risk are pure math — they finish instantly. The NPS step translates non-English comments using GPT — you can see it found and translated comments. The CSM analysis is the slowest step because it makes one LLM call per account — about 26 calls total. This takes around 30 to 60 seconds.

(wait for Step 4) Step 4 scored all accounts. You can see the distribution — how many are High, Medium, and Low risk.

(wait for Step 5) Step 5 generated plain-English explanations for the at-risk accounts.

(wait for Step 6) And Step 6 discovered non-obvious insights.

(results appear)

Now look at the Risk Summary panel. It shows the total accounts analyzed, how many are in each tier, and the total ARR — Annual Recurring Revenue — that’s at risk.

Look at the table — it’s sorted by risk score, highest first. You can see each account’s name, risk percentage, tier, ARR, days until renewal, SDK version, and what’s driving their risk.

And here are the Non-Obvious Insights. The LLM found several patterns — let me highlight the most interesting ones.”

PART 5: NON-OBVIOUS INSIGHTS — THE KEY DIFFERENTIATOR (7:00 - 9:00)

| [SHOW: Terminal output — scroll to the Insights panel]

SAY:

“The assignment specifically asked for insights that a simple rule-based system would miss. Let me walk through what my system found.

Insight 1 — Silent Churn. This is the most dangerous pattern. Some accounts have decent NPS scores — 7 or 8 — which a rule-based system would mark as ‘fine’. But when you cross-reference with usage data, their usage is actually declining significantly. They like our PEOPLE, not our PRODUCT. They’re being polite in surveys while quietly migrating away. The classic example from the data is Meridian Health — NPS of 8, but CSM notes literally say ‘classic silent churn pattern’ because they built a homegrown replacement.

Insight 2 — SDK Deprecation Cascade. This connects three data sources that would normally be analyzed separately. The changelog says SDK v3 is being killed. The usage data shows which accounts are still on v3. And the ticket data shows these SAME accounts have disproportionately more support tickets. The chain is: deprecation announcement causes migration difficulty, which causes tickets, which causes frustration, which causes churn. A rule-based system checking each source independently would never see this chain.

Insight 3 — Champion Loss as a Leading Indicator. Some accounts have perfectly stable metrics right now — good usage, decent NPS. But the CSM notes reveal that their internal champion is leaving or losing influence. For example, Orion Education — their champion is nervous about her role in a post-merger restructuring. Once she’s gone, nobody fights to keep Contentstack. This is a LEADING indicator that predicts a drop 1-2 months before the numbers show it.

Insight 4 — NPS Score-Sentiment Contradictions. Some accounts gave a high NPS score but wrote a negative comment, or vice versa. For instance, a score of 8 but the comment says ‘execution has fallen off a cliff’. A rule-based system trusting the number alone would miss this. My system detects the keyword mismatch and boosts the risk score.

You can also see the structured insight counts at the bottom — how many accounts were flagged for each pattern.”

PART 6: STREAMLIT DASHBOARD (9:00 - 11:00)

[TYPE in terminal:] `streamlit run app.py` [Browser opens — show the dashboard]

SAY:

“Now let me show you the interactive dashboard. This is built with Streamlit and Plotly.

(Tab 1 — Risk Overview) The first tab shows the Risk Overview. You can see a pie chart of risk tier distribution, a bar chart showing ARR by risk tier — this tells you how much MONEY is at risk, not just how many accounts. And this scatter plot maps every account by risk score vs ARR — the bigger and redder the dot, the more urgent it is.

Below is the full table of all accounts ranked by risk, and you can see the sidebar filters — you can filter by risk tier, plan tier, or minimum ARR.

(Tab 2 — Account Deep-Dive) The second tab is the Account Deep-Dive. Let me select one of the high-risk accounts. You can see the risk score, tier, ARR, and days until renewal at the top. Below is the AI Risk Assessment — this is the LLM-generated explanation. Then there’s a horizontal bar chart showing which signals contributed most. And on the right, you see the detailed information — NPS score, tickets, competitor mentions, champion status.

(Tab 3 — Signal Analysis) The third tab shows a signal heatmap — at-risk accounts on the Y axis, the 6 signals on the X axis, color-coded from green to red. This lets you visually spot which signals are the problem for each account. You can also see SDK version distribution and plan tier distribution vs risk.

(Tab 4 — *Non-Obvious Insights*) And the fourth tab shows all the non-obvious insights — both the LLM-discovered patterns and the structured pattern detection results with details for each flagged account.”

PART 7: OUTPUT FILES (11:00 - 11:30)

| [SHOW: output/ folder in VS Code, open one CSV briefly]

SAY:

“The pipeline also exports everything to the output folder. We have 4 files:

- risk_scored_accounts.csv — every account with their composite score and tier
- detailed_signals.csv — the full breakdown of all 6 signal scores
- account_briefings.json — the LLM-generated explanations for every account
- insights.json — the summary statistics and non-obvious insights

These can be imported into Salesforce, shared with the sales team, or used as input for downstream automation.”

PART 8: DESIGN DECISIONS & TRADEOFFS (11:30 - 12:30)

| [SHOW: README.md — scroll to Design Decisions section]

SAY:

“Let me quickly explain my key design decisions.

Why these weights? I gave Usage Decline and CSM Sentiment the highest weight at 25% each — because usage is hard data showing disengagement, and CSM notes contain the earliest qualitative signals like competitor mentions. SDK risk and tickets get 15% because they’re important but more situational. NPS only gets 10% because, as we saw, it can be misleading.

Why GPT-4o-mini? It’s the most cost-effective model in the GPT-4 family — about 2 to 5 cents per full pipeline run. It’s fast, and the quality is good enough for structured extraction and explanation generation.

Why is the LLM not a gimmick? I use it in 4 specific places where regex or rules genuinely cannot work — translating Chinese and French comments, understanding messy human writing in CSM notes, synthesizing 6 signal dimensions into a coherent narrative, and finding cross-cutting patterns across 120 accounts. These are real NLP tasks, not cosmetic use.

What I’d do with more time: - Train an actual ML model on historical churn data to learn the signal weights empirically - Add time-series forecasting to predict WHEN usage will drop below critical thresholds - Build automated Slack alerts when an account

crosses a risk threshold - Add a feedback loop where account managers can mark predictions as correct or incorrect - And most importantly — add unit tests and deploy with Docker for production use.”

PART 9: CLOSING (12:30 - 13:00)

| [SHOW: GitHub repo page]

SAY:

“To summarize — this project ingests 5 messy data sources, reconciles inconsistent names using fuzzy matching, extracts 6 risk signals using both math and AI, scores every account on a 0-to-100 scale, generates actionable explanations using LLM, and discovers non-obvious patterns that a rule-based system would miss.

The entire pipeline runs in about 60 to 90 seconds and costs less than 5 cents per run.

The code is on GitHub at github.com/sohammmmm10/renewal-intelligence. Thank you for reviewing my submission.”

TIPS FOR RECORDING

1. **Use Loom** (free) — it’s the easiest. Go to loom.com, install the Chrome extension, hit Record
2. **Screen + Camera** — show your face in the corner if possible, makes it more personal
3. **Don’t rush** — speak naturally, it’s okay to pause
4. **If you make a mistake** — just keep going, don’t restart. Nobody expects perfection
5. **Total time target:** 10-13 minutes (not longer)
6. **Before recording:** Run `python run_pipeline.py` once so the output is fresh. Also make sure `streamlit run app.py` works

Commands you’ll need during recording:

```
cd "C:\Users\soham.dahivalkar\Downloads\renewal_intelligence_takehome
(1)\renewal-intelligence"
.\venv\Scripts\activate
python run_pipeline.py
streamlit run app.py
```